

QVM

September 19, 2026

Week-03(Day-10):Quantum Support Vector Machine (QSVM);RAJIV KUMAR YADAV

This program demonstrates a basic Quantum Support Vector Machine (QSVM) using Qiskit and Qiskit Aer. It classifies a small two-dimensional dataset into two classes, -1 and $+1$, using a quantum kernel.

```
[1]: import qiskit
import numpy as np

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

print("Qiskit:", qiskit.__version__)
print("NumPy:", np.__version__)

# =====
# 1. DATASET
# =====

X = np.array([
    [0.1, 0.2],
    [0.2, 0.1],
    [2.8, 2.9],
    [2.9, 2.8]
])

# Two classes: -1 and +1
y = np.array([-1, -1, 1, 1])

# Scale features to rotation angles
X_scaled = (X / 3.0) * np.pi

# =====
# 2. QUANTUM FEATURE MAP
# =====

def feature_map(x):
```

```

qc = QuantumCircuit(2)

# Encode classical features
qc.ry(x[0], 0)
qc.ry(x[1], 1)

# Create entanglement
qc.cx(0, 1)

# Data-dependent rotation
qc.rz(x[0] * x[1], 1)

return qc

# =====
# 3. QUANTUM KERNEL
# =====

simulator = AerSimulator()

def quantum_kernel(x1, x2, shots=2048):

    qc1 = feature_map(x1)
    qc2 = feature_map(x2)

    # Prepare  $|\phi(x1)\rangle$ 
    qc = qc1

    # Apply inverse feature map  $|\phi(x2)\rangle$ 
    qc.compose(
        qc2.inverse(),
        inplace=True
    )

    qc.measure_all()

    compiled = transpile(qc, simulator)

    result = simulator.run(
        compiled,
        shots=shots
    ).result()

    counts = result.get_counts()

```

```

    # Probability of  $|00\rangle$ 
    kernel_value = counts.get("00", 0) / shots

    return kernel_value

# =====
# 4. BUILD QUANTUM KERNEL MATRIX
# =====

n = len(X_scaled)

K = np.zeros((n, n))

print("\nCalculating Quantum Kernel Matrix...")

for i in range(n):
    for j in range(n):

        K[i, j] = quantum_kernel(
            X_scaled[i],
            X_scaled[j]
        )

print("\nQuantum Kernel Matrix:")
print(np.round(K, 3))

# =====
# 5. QSVM CLASSIFIER
# =====

def predict(x):

    kernel_values = []

    for train_x in X_scaled:

        value = quantum_kernel(
            train_x,
            x
        )

        kernel_values.append(value)

    kernel_values = np.array(kernel_values)

```

```

# Similarity with each class
class_minus = kernel_values[y == -1].mean()
class_plus = kernel_values[y == 1].mean()

if class_plus > class_minus:
    return 1
else:
    return -1

# =====
# 6. PREDICTION
# =====

predictions = []

for x in X_scaled:
    predictions.append(
        predict(x)
    )

predictions = np.array(predictions)

# =====
# 7. ACCURACY
# =====

accuracy = np.mean(
    predictions == y
) * 100

print("\n=====")
print("      QUANTUM SVM (QSVM)")
print("=====")

print("Actual labels   :", y)
print("Predicted labels:", predictions)
print("Accuracy         :", accuracy, "%")

```

Qiskit: 2.5.2

NumPy: 2.4.6

Calculating Quantum Kernel Matrix...

```
Quantum Kernel Matrix:
[[1.    0.993 0.    0.   ]
 [0.994 1.    0.    0.   ]
 [0.    0.    1.    0.996]
 [0.    0.    0.993 1.   ]]
```

```
=====
      QUANTUM SVM (QSVM)
=====
Actual labels   : [-1 -1  1  1]
Predicted labels: [-1 -1  1  1]
Accuracy       : 100.0 %
```

[]: